Arthur Torres

Software Engineer | Python | FastAPI | SQL | AWS | Docker
Belo Horizonte, Brazil | arthuroliveiratorres@gmail.com | +55 31 99800-0636
linkedin.com/in/torres-arthur | github.com/tutorres

Summary

Software engineer with over two years of professional work, now in a bank global operations team: Python services and REST APIs, containerized with Docker and shipped through the bank GitLab CI/CD to Kubernetes, with SQL as the data layer. I build LLM agents that read documents and reconcile discrepancies, covered by pytest and TDD. I also build with TypeScript, React and Next.js.

Technical skills

Proficient: Python, SQL, FastAPI, REST APIs, pytest and TDD
Working use: Docker, AWS (S3, IAM, KMS, Parameter Store), Terraform, Kubernetes, GitLab CI/CD, Argo CD, Grafana, PostgreSQL, Trino, TypeScript, React, Next.js
AI: LLM agents, OpenAI API, MCP servers, prompt and context design, structured extraction from documents, Cursor and Claude Code (spec first)
Languages: Portuguese (native), English C1 (EF SET), used daily with the US team

Experience

Banco Inter | Software Engineering Intern, AI and Automation team, Global Operations | Jan 2026 to Present
- Onboarding decision job: every 15 minutes it reads the account queue, normalizes malformed JSON of about 1,200 lines per company from several systems, and returns which rule fired and the fields it fired on. Rules designed with the operations team.
- Document analysis: the job calls a separate API I designed, passing a pre-signed link instead of the file because the documents are sensitive. The API answers 202, runs the document through an LLM agent, posts score and rationale back to the onboarding system and messages the analyst on Slack. A SHA-256 fingerprint stops the same document from calling the model twice. About 12 documents a day, in production.
- Reconciliation API with MCP: gives the LLM a read-only SQL tool over Trino, scoped to a dedicated service user that sees only the tables each reconciliation needs, and returns structured JSON ranking cases by likelihood of real divergence. Modular across reconciliations.
- Negative balance notices: SQL selects the accounts and the job calls the bank messaging API twice a week over ~20 accounts, with dry-run and send flags per environment. Replaces about 12h/week of manual case handling (process owner estimate).
- Models and delivery: I compare the models in the gateway against reference answers kept out of the model call, one variable per run, and track cost in a Grafana dashboard I built. Services written with TDD (~85% coverage), per-environment Kubernetes values files and Terraform resources (Parameter Store, S3, IAM, gateway keys).
- In development: a job that consumes a Salesforce queue through the API the Salesforce team configured, closes the account through an internal production API and then closes the case.
Stack: Python, FastAPI, SQL, Trino, MCP, Docker, Kubernetes, Terraform, AWS, GitLab CI/CD, Argo CD, Grafana, pytest

TMB Scan | Full Stack Developer, contractor, with a partner | Mar 2026 to Sep 2026
- Co-built a live assessment platform for a paying client, on its own domain, deployed on Vercel. I wrote the initial base, the Morph module with vitest tests, the paginated PDF report rendering and the deploy configuration, and co-designed the multi-tenant data model with my partner. Per-user access with Supabase row level security.
Stack: Next.js, React, TypeScript, Tailwind CSS, Supabase (PostgreSQL), vitest

Akyou Solucoes | Software Engineer, contractor, only engineer at a five-person accessibility edtech | Apr 2024 to Dec 2025
- Built the product prototype used in meetings with clients and investors: a FastAPI service that extracts the text of a textbook PDF and runs it through an OpenAI Assistant for the chosen accessibility need, returning structured JSON, and a React and TypeScript app that shows each page with its adaptations, editable before use. Login and history on Supabase.
- Generated only the adaptation the teacher selects, to keep token cost down. Containerized with Docker and deployed on AWS Elastic Beanstalk, learned on my own. Discovery with the founder and prospective clients before building.
Stack: Python, FastAPI, OpenAI API, React, TypeScript, Supabase, Docker, AWS Elastic Beanstalk

Projects

Financial Intelligence Dashboard | github.com/tutorres/Financial-Intelligence-Dashboard
Layered data pipeline on DuckDB with a natural language interface that answers questions in SQL.

Vale Desenvolver 2026, open challenge | github.com/tutorres/Vale_Desenvolver
Predictive maintenance over 37 million rows of mining telemetry, XGBoost on a temporal split benchmarked against baselines. Four data-quality defects fixed, one visible only on the full dataset, with a pytest suite.

Education

CEFET-MG | B.Eng. Computer Engineering, 5th semester, expected 2029
Undergraduate research, CEFET-MG | LLMs applied to financial time series | 2026 to present